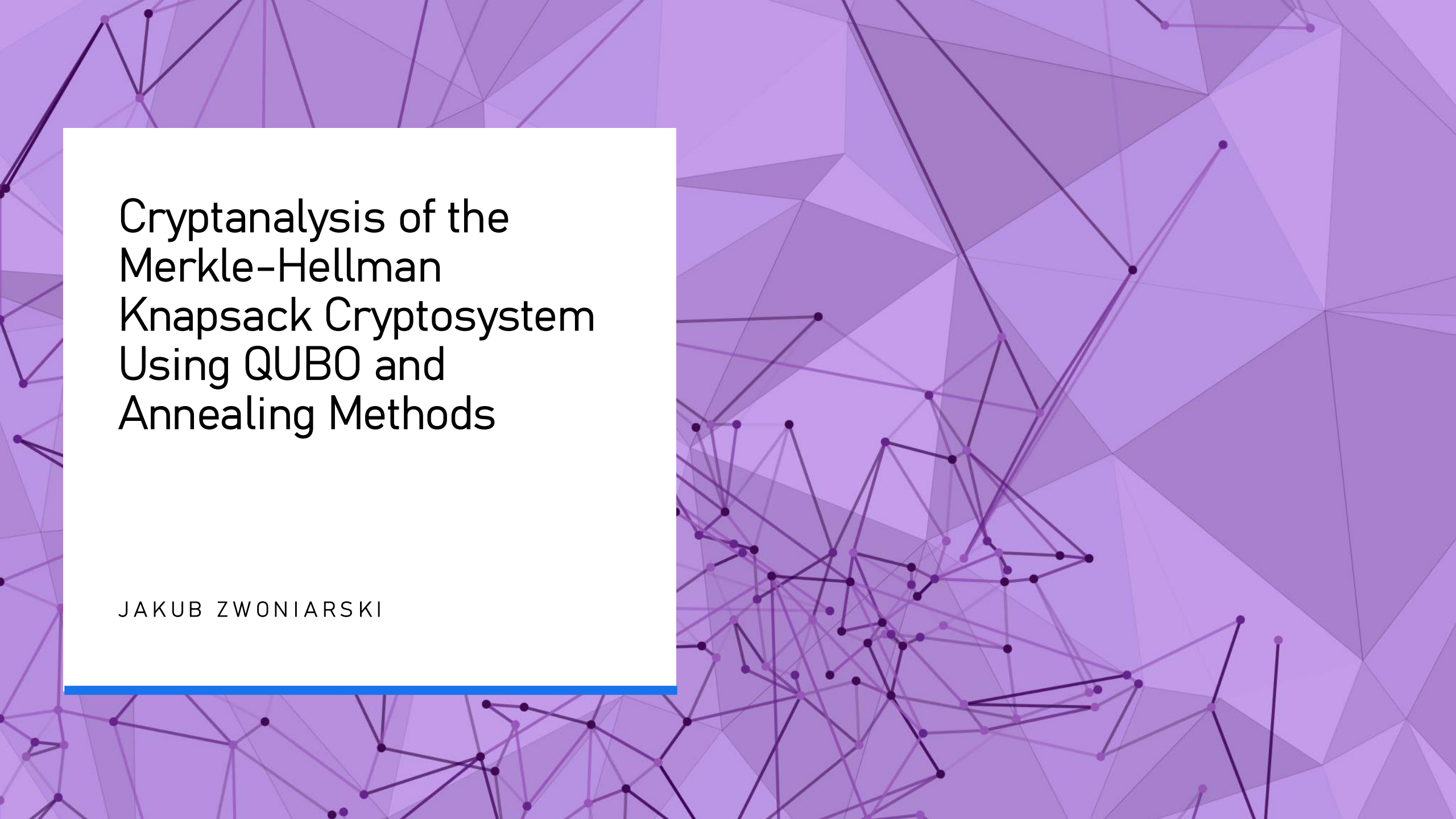




Cryptanalysis of the Merkle-Hellman Knapsack Cryptosystem Using QUBO and Annealing Methods

JAKUB ZWONIARSKI

Why This Topic?

01

subset-sum is a classical hard combinatorial problem

02

Merkle-Hellman is historically based on subset-sum

03

QUBO is a natural format for binary optimization

04

annealing methods can be tested on small instances

Merkle-Hellman Idea

- Given:
- public weights (a_i)
- ciphertext (C)

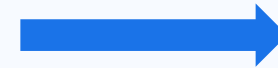
Private key: $w_i > \sum_{j<i} w_j$

Public key: $a_i = rw_i \bmod m$

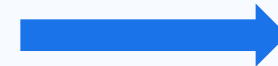
Encryption: $C = \sum_i a_i x_i$

Subset-sum recovery

Public weights a_i
Ciphertext C



Find x in $\{0, 1\}^n$
so sum $a_i x_i = C$



Recovered bit vector
identifies the private subset

QUBO Formulation

- Objective: $E(x) = (\sum_i a_i x_i - C)^2$
- Exact solution: $E(x) = 0$
- Expanded QUBO:

$$C^2 + \sum_i (a_i^2 - 2C a_i) x_i + \sum_{i < j} 2a_i a_j x_i x_j$$

QUBO coefficient structure

q11	q12	q13	q14	Diagonal $a_i^2 - 2C a_i$
	q22	q23	q24	
		q33	q34	Pair terms $2a_i a_j$
			q44	

Objective becomes $x^T Q x$

Solvers

Compared methods:

- Brute force
- Exact QUBO solver
- Simulated annealing
- Local SQA-style solver

How each solver is used

01

Brute force

Exact baseline for small n

02

Exact QUBO

Checks objective formulation

03

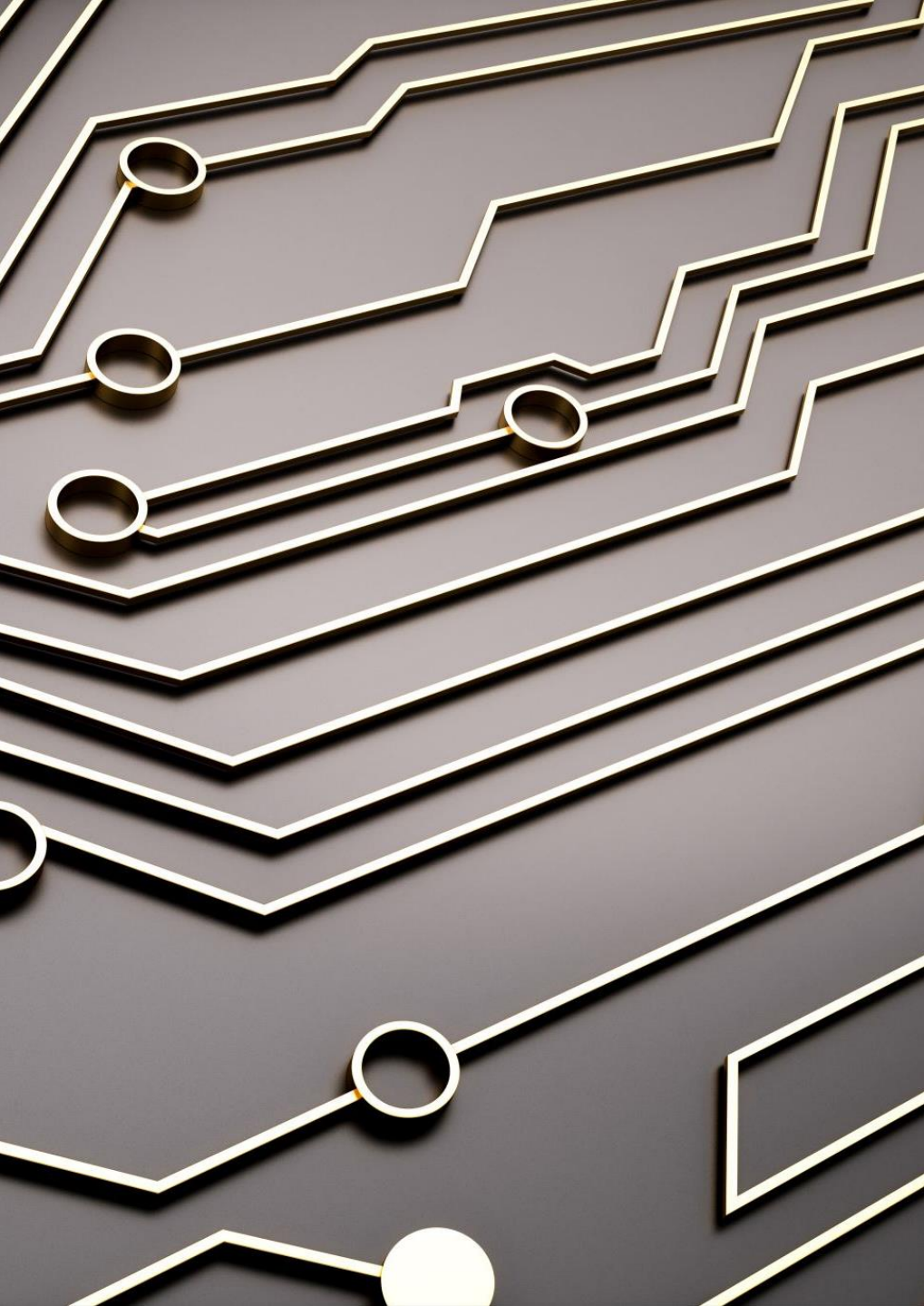
Simulated annealing

Fast heuristic search

04

SQA-style

Local quantum-inspired comparison



Important Distinction: SQA vs QPU

- This project does **not** use a physical D-Wave QPU.

SQA-style solver:

- local simulation
- Trotter replicas
- classical CPU runtime
- quantum-annealing-inspired, not quantum hardware

Implementation Structure

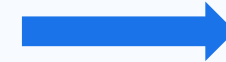
Repository structure:

- merkle_hellman.py
- subset_sum.py
- qubo.py
- solvers/
- experiments/
- notebooks/
- report/

Module flow

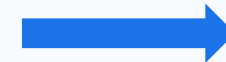
1
Model

merkle_hellman.py
subset_sum.py



2
QUBO

qubo.py builds $E(x)$ and Q



3
Run

solvers/ + experiments/
produce benchmark outputs

Validation

Validation checks:

- Tests: 48 passing
- QUBO energy equals objective
- Exact hit means $E(x)=0$
- Benchmark grid matches config
- Planted match implies exact hit

Validation gate

48

tests passing

$E(x)$

matches objective

0

means exact hit

Grid

matches config

If any gate fails, benchmark results are not trusted

Experimental Setup

Benchmark:

- Sizes: $n = 8, 12, 16$
- Trials: 5
- Sources: random, Merkle-Hellman
- Solvers: brute force, SA, SQA-style
- Metrics: exact hit, planted match, runtime, objective, Hamming distance

Benchmark design

Instance size	$n = 8, 12, 16$
Source	Random + Merkle-Hellman
Solver set	Brute force / SA / SQA
Metrics	Hit, runtime, objective, distance

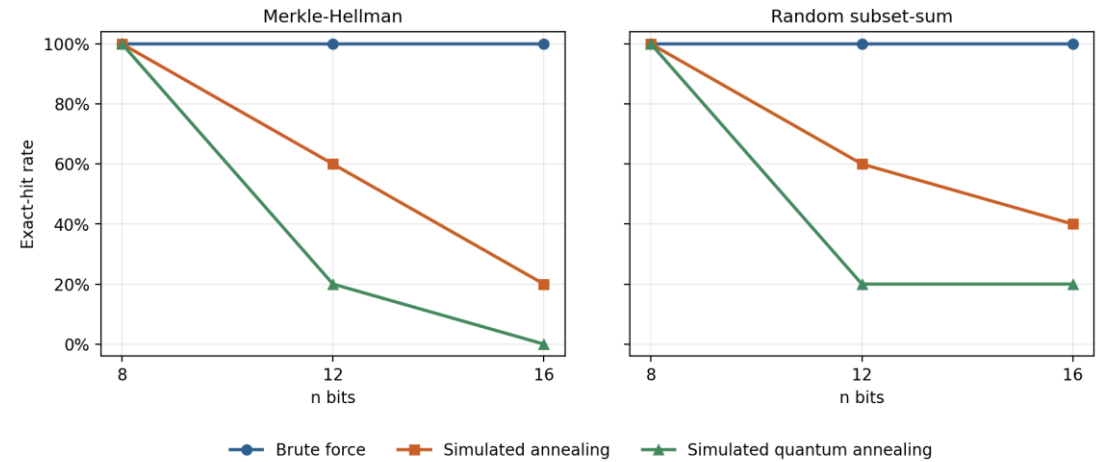
5 trials per condition

Results: Success and Runtime

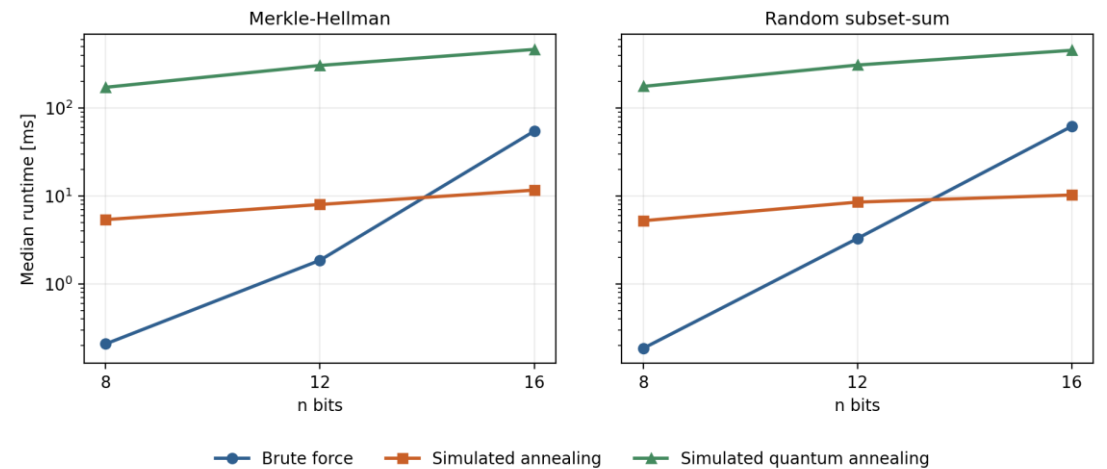
Main observations:

- Brute force: always exact
- SA: fast, but success drops
- SQA-style: slower and not better in this setup

Exact subset-sum recovery



Runtime by instance size

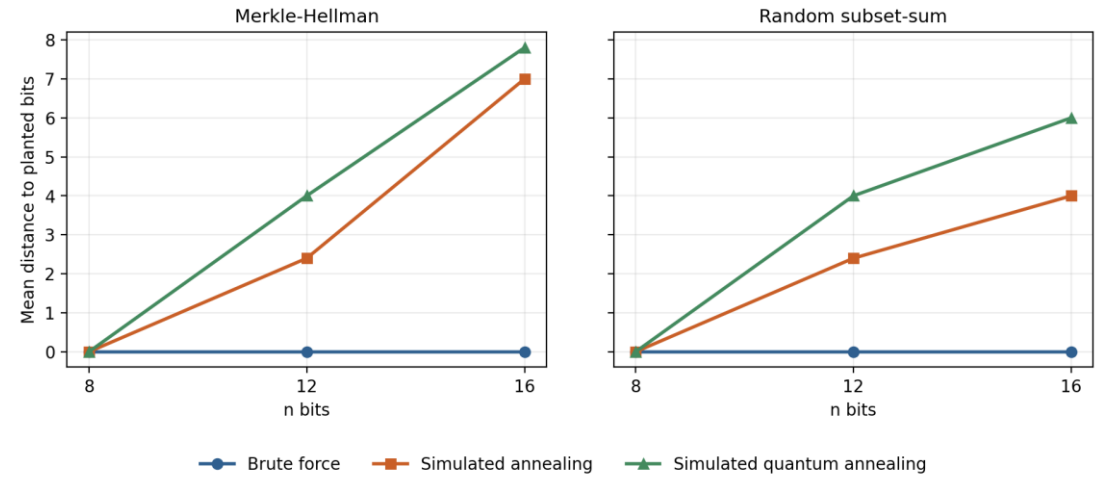


Results: Objective and Hamming Distance

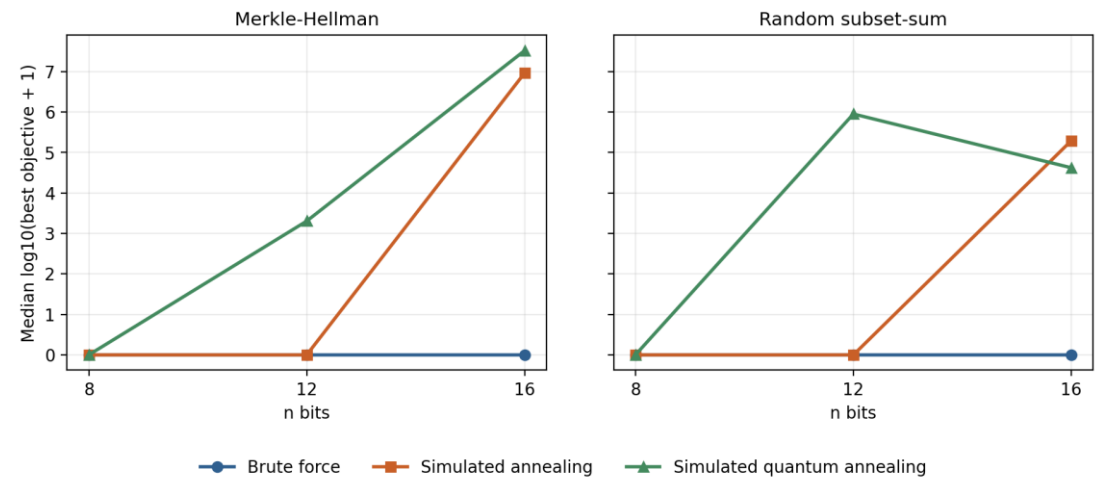
Main observations:

- Nonzero objective means missed subset-sum target
- Hamming distance ($d_H(\mathbf{x}, \hat{\mathbf{x}}) = \sum_{i=1}^n |x_i - \hat{x}_i|$) shows bit-level recovery quality
- Heuristic failures become larger at $n = 16$

Recovered bit-vector distance



Residual objective gap





Conclusions

Main takeaways:

- Merkle-Hellman recovery can be modeled as subset-sum
- QUBO correctly represents the objective
- Brute force validates small cases
- SA is fast but heuristic
- Local SQA-style solver does not outperform SA here
- Project is educational and reproducible



Possible Questions

- Why not test larger instances?
- Is this a real attack on modern cryptography?
- Why use QUBO?
- Why can exact-hit rate differ from planted-match rate?
- What are Trotter replicas in your SQA-style solver?
- What is the decreasing transverse-field-like schedule?